

SANITEK

Guía de Arquitectura Backend · Quarkus Microservicio

■ Este documento describe la arquitectura de paquetes propuesta y corregida para el backend de Sanitek en Quarkus. Los cambios marcados con ♦ son adiciones o modificaciones respecto a la propuesta inicial v1.0, justificadas técnicamente en la sección de cambios.

1. Estructura de Paquetes Corregida

Paquete / Carpeta	Responsabilidad
<code>org.acme/domain/entity/</code>	Entidades JPA — extienden <i>BaseEntity</i> (<i>id</i> , <i>createdAt</i> , <i>updatedAt</i>)
<code>org.acme/domain/enums/</code>	Enumeraciones del dominio
<code>org.acme/domain/irsa/ ♦ NUEVO</code>	Lógica pura del cálculo IRSA (sin dependencias de infra). Aquí vive el motor del índice.
<code>org.acme/dto/request/</code>	Datos de entrada POST/PUT — <i>@NotNull</i> , <i>@Size</i> , <i>Bean Validation</i>
<code>org.acme/dto/response/</code>	Records inmutables de Java para la salida al cliente
<code>org.acme/repository/</code>	Acceso a datos — <i>Panache Repositories</i>
<code>org.acme/service/</code>	Lógica de negocio <i>@ApplicationScoped</i> — operaciones escritura con <i>@Transactional</i>
<code>org.acme/mapper/ ♦ MOVIDO</code>	Conversión <i>Entity</i> ↔ <i>DTO</i> . Vive al mismo nivel que <i>service/</i> , NO dentro de <i>rest/</i>
<code>org.acme/rest/</code>	Controladores HTTP — <i>JAX-RS Resources</i> . Solo llama a <i>service/</i> y <i>dto/</i>
<code>org.acme/infrastructure/auth/ ♦ NUEVO</code>	Interfaz <i>AuthProvider</i> en <i>domain/</i> + implementación <i>Firebase</i> aquí
<code>org.acme/infrastructure/datasource/ ♦ NUEVO</code>	Clientes HTTP: <i>SinaicaClient</i> , <i>EraClimateClient</i> , <i>InegiClient</i> , <i>ConacytClient</i>
<code>org.acme/infrastructure/scheduler/ ♦ NUEVO</code>	<i>Jobs @Scheduled</i> para actualización periódica del IRSA
<code>org.acme/config/</code>	<i>@ConfigMapping</i> / <i>@ConfigProperties</i>
<code>org.acme/exception/</code>	<i>AppException</i> + <i>GlobalExceptionHandler</i>

Paquete / Carpeta	Responsabilidad
org.acme/util/	Utilidades transversales

Los paquetes marcados con **◆ NUEVO** o **◆ MOVIDO** son los cambios respecto a la propuesta v1.0. El resto se mantiene igual.

2. Capas y Reglas de Dependencia

rest/	service/, dto/, mapper/	repository/, domain/entity/ directamente
service/	repository/, domain/, mapper/, infrastructure/	rest/, dto/request/
mapper/	domain/entity/, dto/	service/, rest/, repository/
repository/	domain/entity/	service/, rest/
infrastructure/	SDKs externos, domain/ (interfaces)	Capas de negocio internas directamente
domain/irsa/	domain/entity/, domain/enums/	Todo lo demás — lógica pura sin dependencias

3. Cambios Respecto a la Propuesta v1.0

Cambio	Motivo	Impacto
mapper/ sube un nivel (fuera de rest/)	El service es quien convierte Entity→DTO, no el Resource HTTP	Evita lógica de transformación en la capa de presentación
domain/irsa/ nuevo paquete	El cálculo del IRSA es el core de negocio; necesita aislamiento	Testeable sin contexto Quarkus, reutilizable entre servicios
infrastructure/auth/ con interfaz AuthProvider	Desacoplar Firebase del dominio; mañana puede ser Keycloak	Cambio de proveedor auth sin tocar lógica de negocio
infrastructure/datasource/ para APIs externas	SINAICA, ERA5, INEGI necesitan clientes HTTP propios	Centraliza manejo de errores y reintentos por fuente
infrastructure/scheduler/ con @Scheduled	El IRSA debe actualizarse periódicamente desde fuentes oficiales	Datos siempre frescos sin intervención manual

4. Convenciones de Implementación

BaseEntity	Todas las entidades extienden BaseEntity: id (UUID), createdAt, updatedAt automáticos
Response = record	Java records — inmutables, sin setters, ideal para DTOs de salida
Bean Validation	Requests con @NotNull, @Size, @Email, @Pattern en todos los campos
@Transactional	Solo en métodos de service/ que realizan escrituras (create, update, delete)
AppException	Excepción de negocio con código HTTP; GlobalExceptionHandler la serializa a JSON
AuthProvider (interfaz)	Definida en domain/, implementada en infrastructure/auth/FirebaseAuthProvider
@Scheduled	Jobs en infrastructure/scheduler/ para consumir APIs externas y recalculer IRSA

5. Puntos de Decisión Pendientes

■ ¿Quién hace la conversión Entity → DTO?

Debe ser el **service/**, no el resource. El resource solo llama al service y devuelve el Response directamente. El mapper/ es una dependencia del service, no del rest/.

■ ¿El cálculo IRSA en un solo servicio o módulo propio?

Con múltiples variables (calidad de aire, INEGI, CONEVAL, ERA5), un IrsaCalculationService monolítico se vuelve un 'God Service'. Recomendamos domain/irsa/ con clases como IrsaEngine, IrsaWeightConfig y IrsaResult (record).

■ ¿Cómo actualizamos el IRSA periódicamente?

Usar @Scheduled en infrastructure/scheduler/IrsaRefreshJob.java. Este job llama a los datasource clients, transforma los datos y delega a IrsaCalculationService para recalculer y persistir los índices. Definir la frecuencia en application.properties.